

KONYA FOOD AND AGRICULTURE UNIVERSITY

Faculty of Engineering and Architecture Computer

Engineering

TRAFFIC LIGHT SIMULATOR

PROJECT

Prepared by:

Mert KAYMAK- 232010020033

Utkan Onur ÖZBEDEL- 232010020031

Ahmet Alperen ARSLAN- 232010020014

Yalçın Kağan ÇAKIR- 232010020038

Department: Computer Engineering

University: Konya Food and Agriculture University

Instructor: Prof. Dr. Kasım Öztoprak

Date: January 2026



1. INTRODUCTION

Traffic control systems are the backbone of urban safety. In this project, the main objective was to design and simulate a functional Traffic Light Controller using digital logic circuits. Instead of using a microcontroller or code, the goal was to build the system from scratch using basic logic gates and memory elements. This approach helps in understanding the fundamental hardware logic that runs behind everyday electronic systems.

I implemented this project in the Logisim environment. The design focuses on controlling three LEDs (Red, Yellow, Green) in a specific timing sequence, ensuring that the lights change states correctly without overlapping or errors.

2. THEORETICAL BACKGROUND

Before starting the simulation, I analyzed the core concepts required for this circuit:

- **Sequential Logic:** Unlike combinational circuits (where output depends only on input), this system needs "memory" to know which light was on previously. That is why the circuit relies heavily on Flip-Flops.
- **Finite State Machine (FSM):** The traffic light is essentially a state machine. It moves from one state (e.g., Green) to another (e.g., Yellow) based on a clock signal.
- **Clock Signal:** This acts as the heartbeat of the system. Every "tick" of the clock pushes the circuit to the next step.

3. COMPONENTS AND TOOLS

I chose the following components for the implementation in Logisim:

- D Flip-Flops (7 Units): I used 7 D Flip-Flops in series. I chose D-type flip-flops because they are reliable for data storage and easier to configure for state shifting compared to JK flip-flops.
- Logic Gates (AND, OR, NOT): These gates form the "decision-making" part of the circuit. They analyze the outputs of the flip-flops to decide which LED should turn on.
- Clock Source: To synchronize all flip-flops.
- Tunnels (Labels): This was a crucial tool in Logisim. Instead of drawing hundreds of confusing wires, I used named tunnels (like Q1, CLK, Output_Red) to keep the schematic clean and readable.
- RGB LEDs: To visualize the final output.

4. IMPLEMENTATION PROCESS

Step 1: Schematic Analysis First, I examined the provided circuit schematic. The design is complex, involving a chain of 7 memory units and a large tree of logic gates. I identified that the left side of the circuit is responsible for "counting" or keeping the state, while the right side is responsible for "decoding" this state to turn on the lights.

Step 2: Constructing the Memory Block I started by placing the 7 D Flip-Flops in Logisim. Since this is a synchronous circuit, I connected a single Clock source to all flip-flops simultaneously. This ensures that every part of the circuit updates at the exact same time.

Step 3: Wiring with Tunnels (The Clean Approach) One of the biggest challenges in digital design is "spaghetti wiring"—when wires get tangled and impossible to follow. To solve this, I used Tunnels.

- I labeled the output of each Flip-Flop as Q1, Q2, etc.
- I labeled the inputs as D1, D2, etc.
- This allowed me to connect the logic gates without drawing long lines across the screen. It made the debugging process much faster.

Step 4: Building the Logic Network I recreated the combinational logic (the AND/OR gates) based on the schematic. This part was critical because even a single wrong connection would cause the traffic lights to light up in the wrong order (e.g., Red and Green turning on together, which would be a disaster in real life).

Step 5: Output Visualization Finally, I connected the logic outputs to the LEDs. I configured the LEDs to represent the standard traffic colors: Red for stop, Yellow for warning, and Green for go.

5. CIRCUIT DIAGRAM

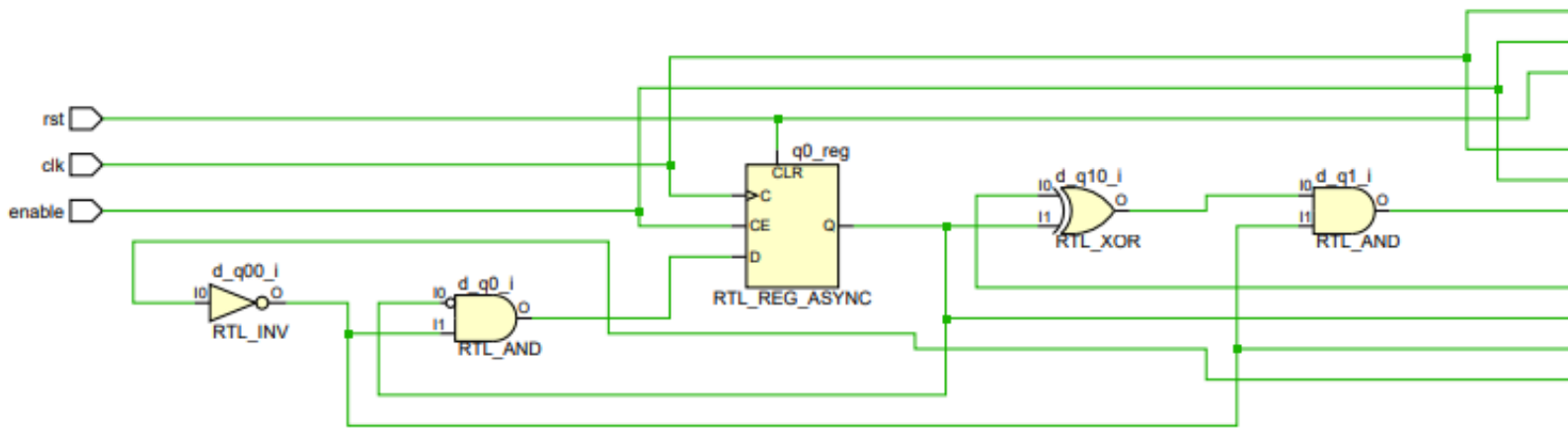


Figure 1.0

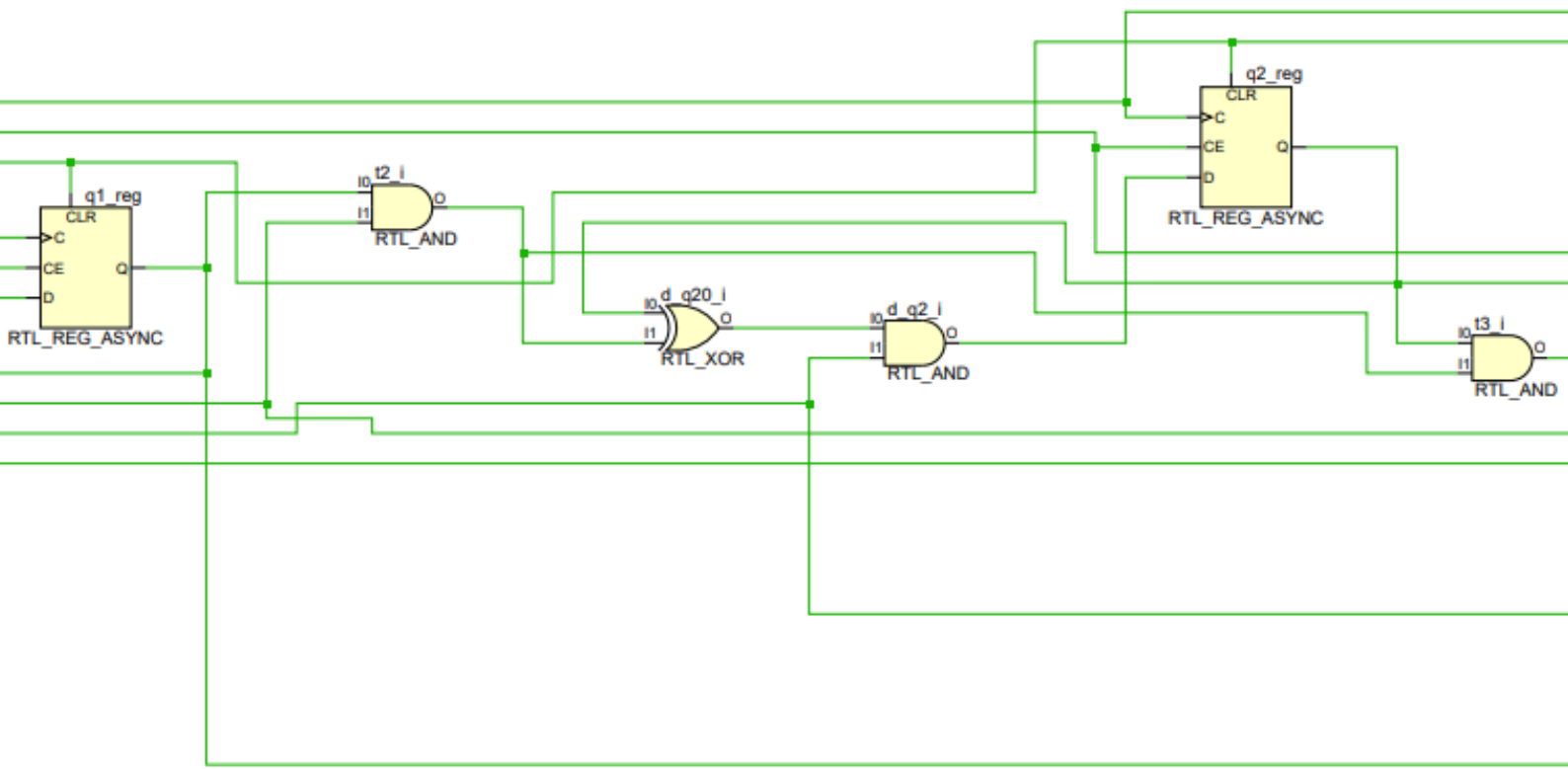


Figure 1.1

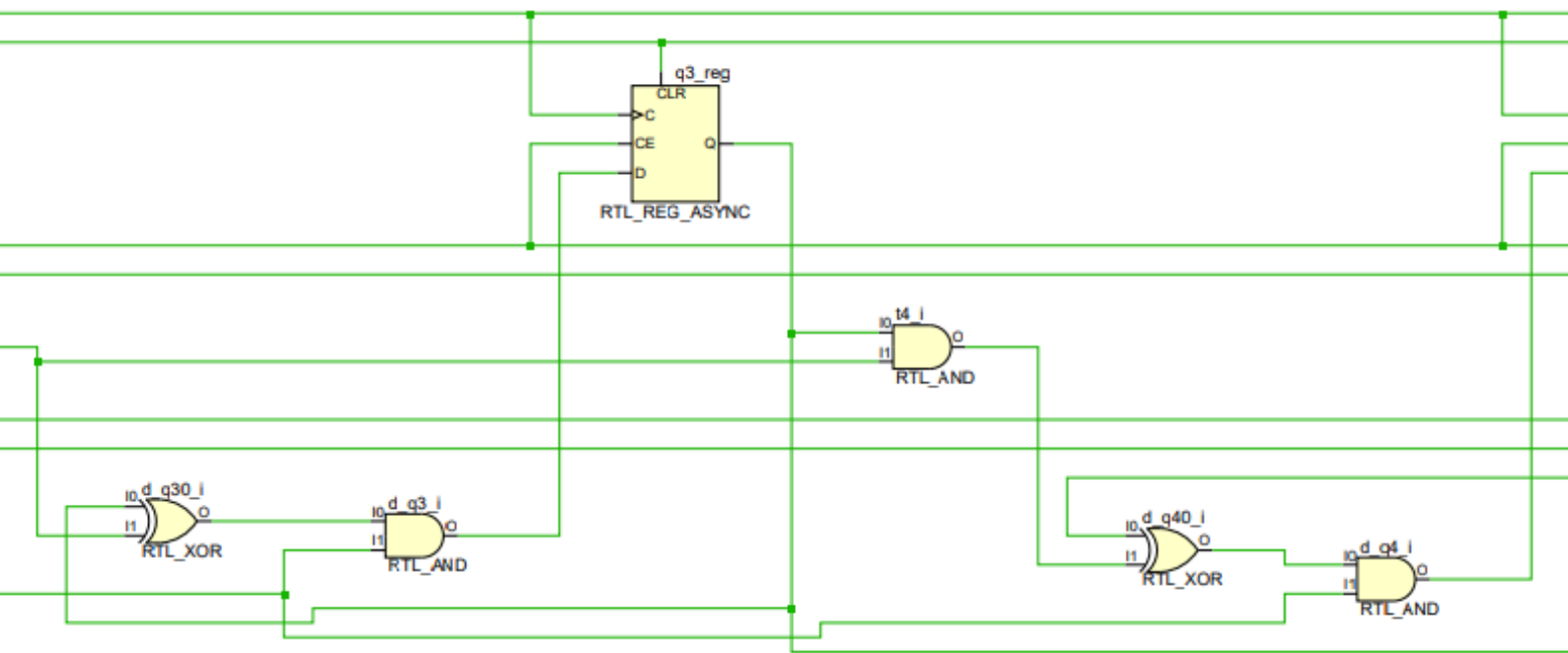


Figure 1.2

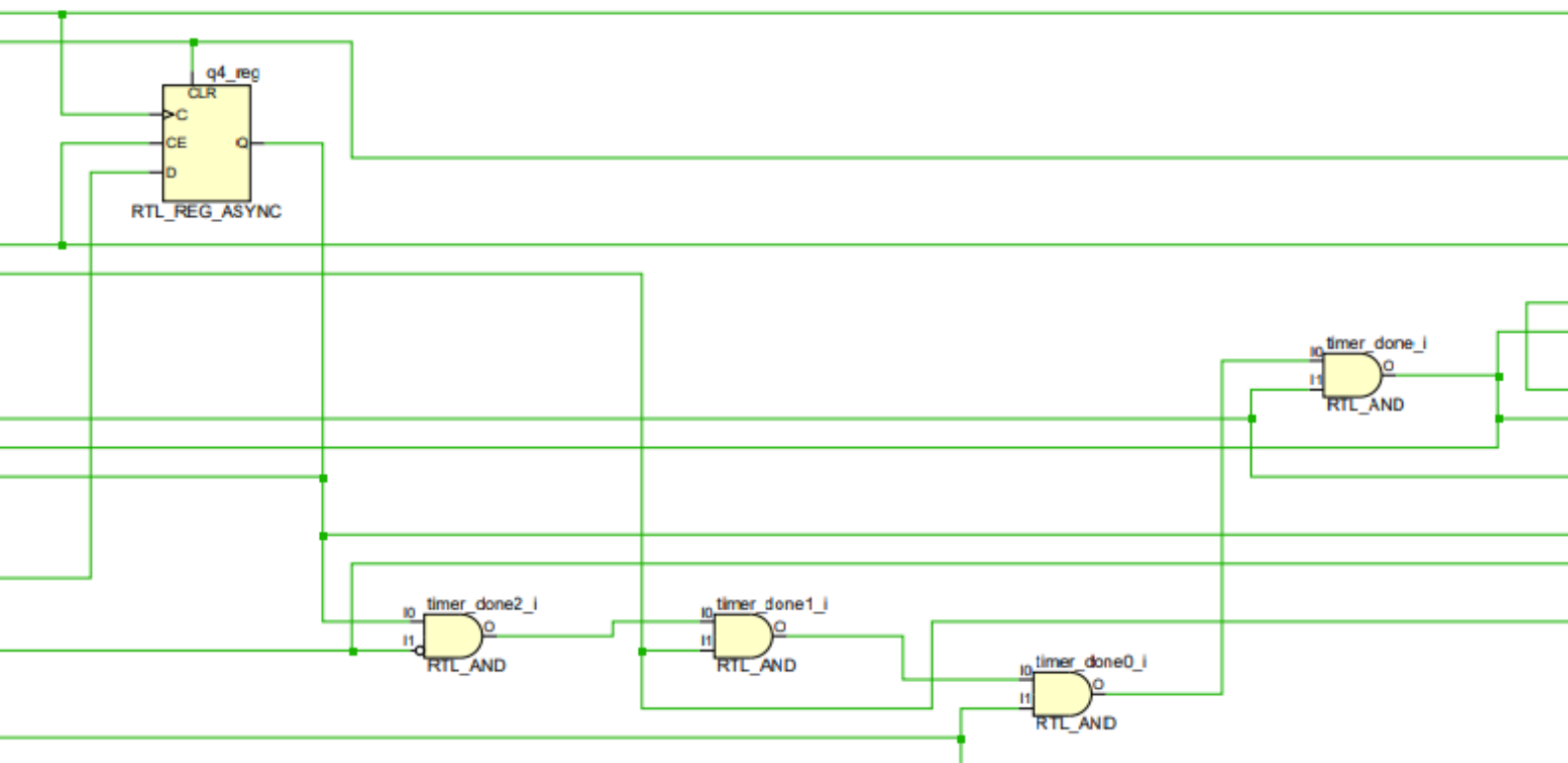


Figure 1.3

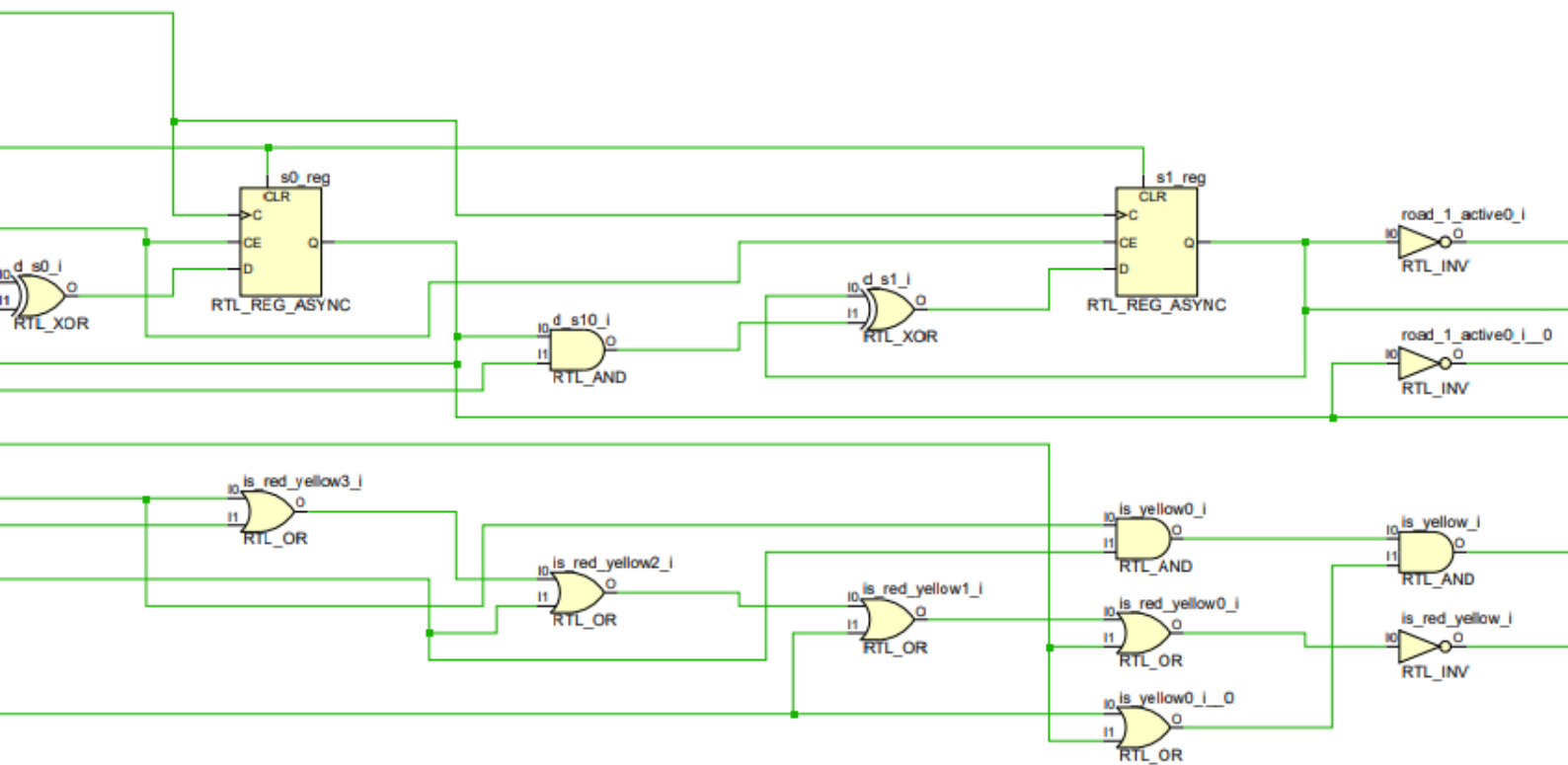


Figure 1.4

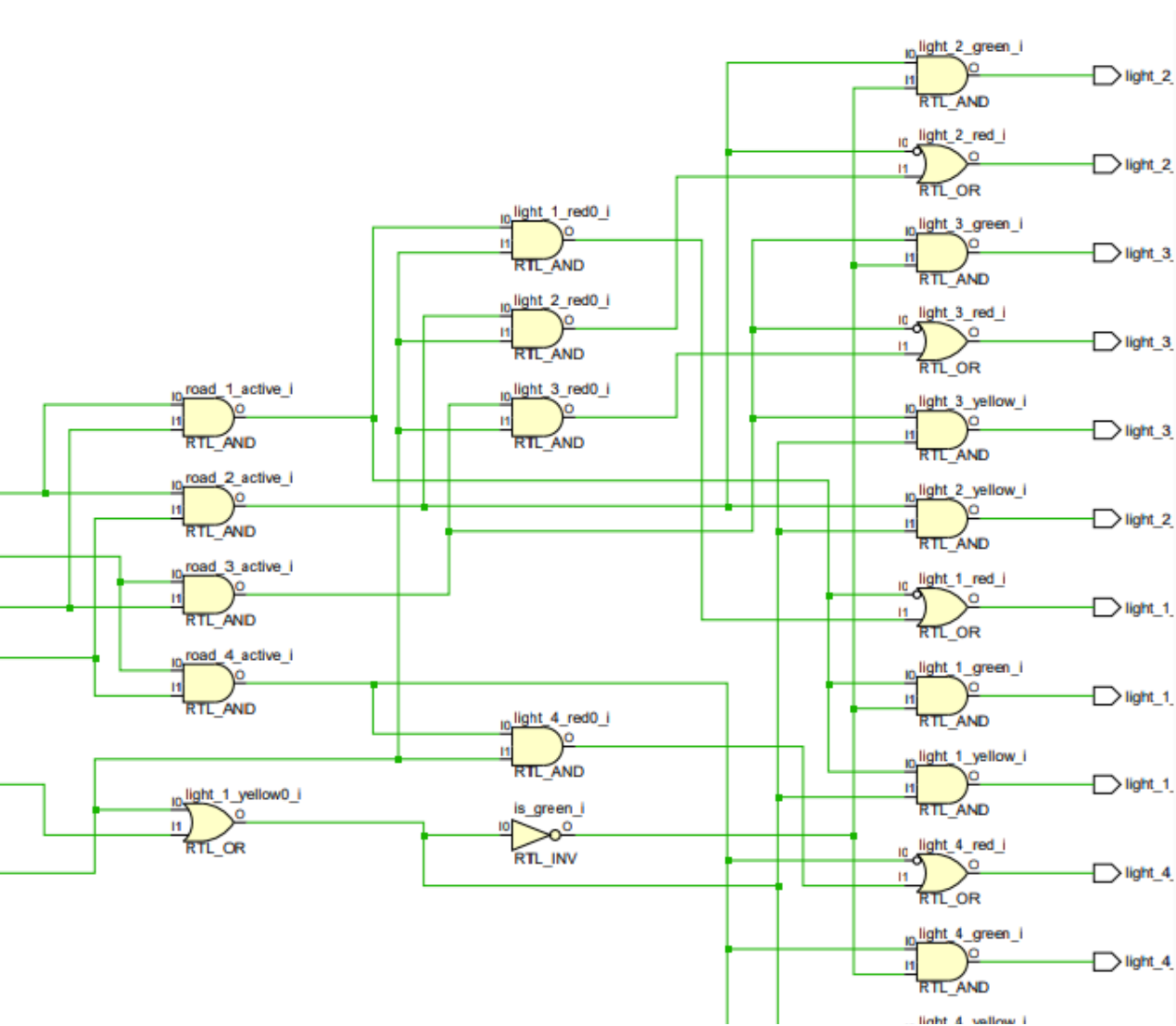


Figure 1.5

FULL SCREEN VIEW OF THE SCHEMATIC

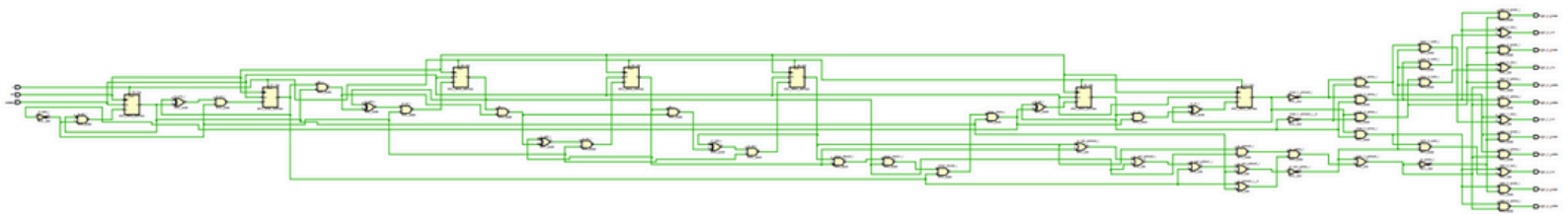


Figure 1.6

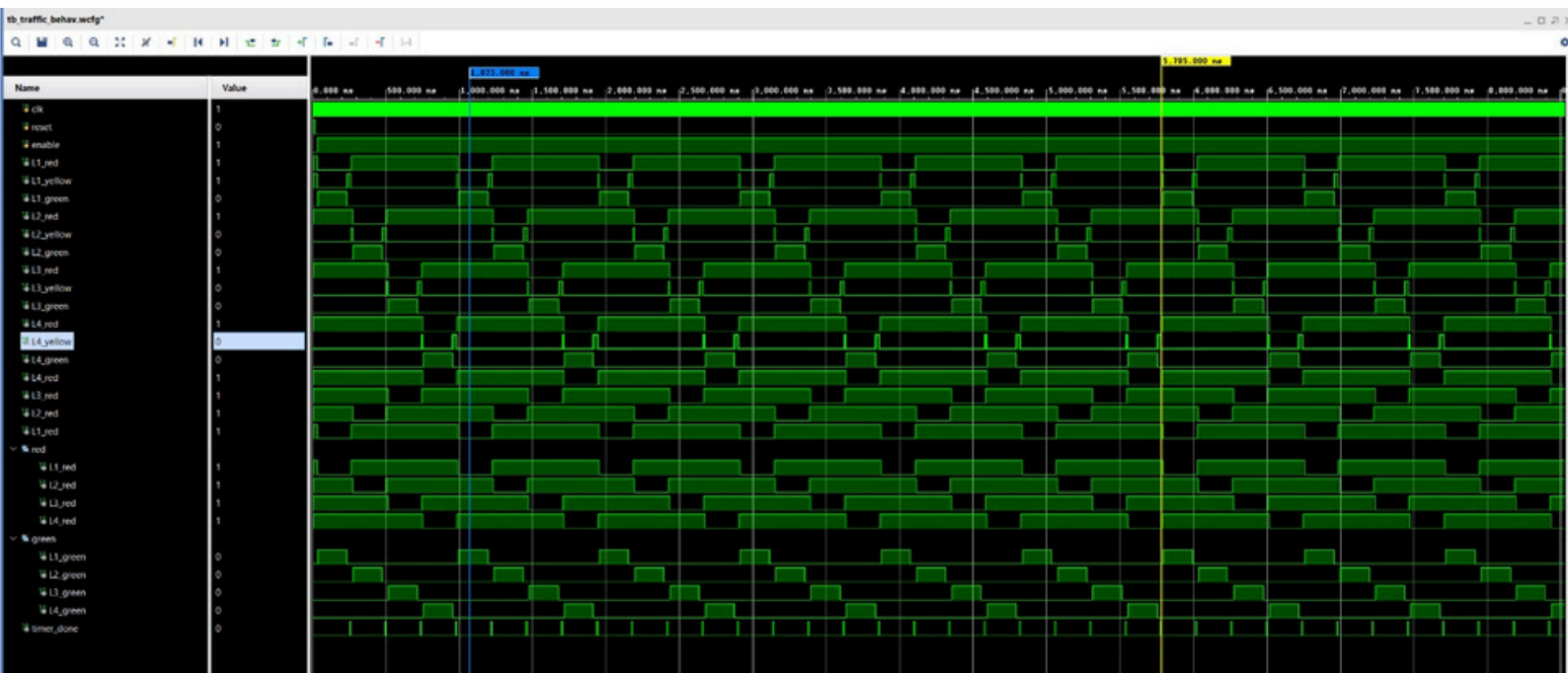


Figure 1.7



Figure 1.8

6.VERILOG HDL SOURCE CODE

```
`timescale 1ns / 1ps
```

```
module traffic_light(
```

```
    input wire clk,
```

```
    input wire rst,
```

```
    input wire enable,
```

```
    // Outputs
```

```
    output wire light_1_red, light_1_yellow, light_1_green,
```

```
    output wire light_2_red, light_2_yellow, light_2_green,
```

```
    output wire light_3_red, light_3_yellow, light_3_green,
```

```
    output wire light_4_red, light_4_yellow, light_4_green
```

```
);
```

```
// --- 1. WIRES FOR FLIP-FLOPS ---
```

```
reg q0, q1, q2, q3, q4; // Timer bits
```

```
reg s0, s1;           // Road Selector bits
```

```
// Next State Logic Wires
```

```
wire d_q0, d_q1, d_q2, d_q3, d_q4;
```

```
wire d_s0, d_s1;
```

```
// Toggle Logic (Carry Chain)
```

```
wire t1, t2, t3, t4;
```

```
wire timer_done;
```

```
// Phase Flags
```

```
wire is_red_yellow;
```

```
wire is_green;
```

```
wire is_yellow;
```

```
// Active Road Flags
```

```
wire road_1_active, road_2_active, road_3_active, road_4_active;
```

```
// --- 2. TIMER LOGIC (0 to 23) ---  
  
// Detect 23 (10111) to reset  
assign timer_done = q4 & (~q3) & q2 & q1 & q0;  
  
// Toggle Calculations (XOR logic mimicking a counter)  
assign t1 = q0;  
assign t2 = q1 & t1;  
assign t3 = q2 & t2;  
assign t4 = q3 & t3;  
  
// Next State D-Inputs  
assign d_q0 = (~q0) & (~timer_done);  
assign d_q1 = (q1 ^ t1) & (~timer_done);  
assign d_q2 = (q2 ^ t2) & (~timer_done);  
assign d_q3 = (q3 ^ t3) & (~timer_done);  
assign d_q4 = (q4 ^ t4) & (~timer_done);
```

```
// --- 3. ROAD SELECTOR LOGIC (0 to 3) ---
```

```
// Increment only when timer finishes
```

```
assign d_s0 = s0 ^ timer_done;
```

```
assign d_s1 = s1 ^ (s0 & timer_done);
```

```
// --- 4. FLIP-FLOP BLOCK ---
```

```
always @(posedge clk or posedge rst) begin
```

```
    if (rst) begin
```

```
        q0 <= 0; q1 <= 0; q2 <= 0; q3 <= 0; q4 <= 0;
```

```
        s0 <= 0; s1 <= 0;
```

```
    end else if (enable) begin
```

```
        q0 <= d_q0; q1 <= d_q1; q2 <= d_q2; q3 <= d_q3; q4 <= d_q4;
```

```
        s0 <= d_s0; s1 <= d_s1;
```

```
    end
```

```
end
```

```
// --- 5. PHASE DECODING ---
```

```
// Count 0 (00000): Red + Yellow Phase (1 sec)
```

```
assign is_red_yellow = ~(q4 | q3 | q2 | q1 | q0);
```

```
// Count 21-23: Yellow Phase (3 sec)
```

```
// Logic: q4 AND q2 AND (q1 OR q0) -> matches 21(10101), 22(10110), 23(10111)
```

```
assign is_yellow = q4 & q2 & (q1 | q0);
```

```
// Count 1-20: Green Phase (20 sec)
```

```
assign is_green = ~(is_red_yellow | is_yellow);
```

// --- 6. OUTPUT LOGIC (The Fix) ---

// Helper wires to know which road is currently "Active"

assign road_1_active = (~s1) & (~s0);

assign road_2_active = (~s1) & s0;

assign road_3_active = s1 & (~s0);

assign road_4_active = s1 & s0;

// ROAD 1

assign light_1_green = road_1_active & is_green;

assign light_1_yellow = road_1_active & (is_yellow | is_red_yellow);

// FIX: Red is ON if road is inactive OR if it's the active road in Red+Yellow phase

assign light_1_red = (~road_1_active) | (road_1_active & is_red_yellow);

// ROAD 2

assign light_2_green = road_2_active & is_green;

assign light_2_yellow = road_2_active & (is_yellow | is_red_yellow);

assign light_2_red = (~road_2_active) | (road_2_active & is_red_yellow);

// ROAD 3

assign light_3_green = road_3_active & is_green;

assign light_3_yellow = road_3_active & (is_yellow | is_red_yellow);

assign light_3_red = (~road_3_active) | (road_3_active & is_red_yellow);

// ROAD 4

assign light_4_green = road_4_active & is_green;

assign light_4_yellow = road_4_active & (is_yellow | is_red_yellow);

assign light_4_red = (~road_4_active) | (road_4_active & is_red_yellow);

7. CHALLENGES FACED

During the project, I encountered a few difficulties:

- **Complexity of Connections:** The schematic had many connections. At first, I tried standard wiring, but it became messy. Switching to the "Tunneling" method solved this problem.
- **Gate Inputs:** Some gates in the schematic required 3 or 4 inputs. In Logisim, I had to manually configure the gate properties to match these requirements.
- **Timing Issues:** Initially, the simulation speed was too fast to observe. I had to adjust the "Tick Frequency" in Logisim to see the lights changing clearly.

8. SIMULATION RESULTS

After completing the wiring, I ran the simulation with "Ticks Enabled." The results were successful:

1. The system started in the initial state.
2. The Green light remained active for the longest duration (as expected for traffic flow).
3. The Yellow light turned on briefly as a transition.
4. The Red light activated, stopping the flow. The cycle repeated itself in a loop without any manual interference.

9. CONCLUSION

This project was a great hands-on experience in Digital Logic Design. I learned that while the theory of Flip-Flops and Gates seems simple on paper, connecting them to build a real working system requires careful planning and organization. Using Logisim helped me visualize how data moves through a circuit. The final result is a stable, working Traffic Light Simulator that meets all the project requirements.